SUPERVISED LEARNING: MODELS & APPLICATIONS

Statistical Foundations and R Implementation

Dr. Lumumba W. Victor

Center for Data Analytics and Modeling
Chuka University

24th March 2026

Resource: `https://cdam.chuka.ac.ke`

Learning Objectives (Part 1)

By the end of this session, participants will be able to:

1. Understand the supervised learning framework and workflow
2. Explain the statistical foundations of key classification models
3. Implement Binary Logistic Regression in R with interpretation
4. Apply K-Nearest Neighbors with appropriate distance metrics

Learning Objectives (Part 2)

5. Formulate SVM optimization and kernel choices
6. Derive Naive Bayes from Bayes' theorem
7. Distinguish LDA vs. QDA assumptions and applications
8. Explain ensemble methods: Random Forest and XGBoost

Session Outline (Part 1)

1. Supervised Learning Framework
2. Data Preparation Recap
3. Evaluation Framework
4. Binary Logistic Regression
5. Logistic: Mathematical Foundation
6. Logistic: Assumptions & Diagnostics
7. Logistic: R Implementation
8. K-Nearest Neighbors Overview
9. KNN: Distance Metrics
10. KNN: Choosing k

Session Outline (Part 2)

11. KNN: R Implementation
12. Support Vector Machines: Concept
13. SVM: Mathematical Foundation
14. SVM: Kernel Functions
15. SVM: R Implementation
16. Naive Bayes: Bayes' Theorem
17. Naive Bayes: Independence Assumption
18. Naive Bayes: R Implementation
19. LDA: Statistical Foundation
20. LDA vs. QDA Comparison

Session Outline (Part 3)

21. LDA/QDA: R Implementation
22. Ensemble Learning: Bagging
23. Random Forest: Theory
24. RF: Feature Importance & OOB
25. RF: R Implementation
26. Boosting: Conceptual Overview
27. XGBoost: Mathematical Foundation
28. XGBoost: Regularization
29. XGBoost: R Implementation
30. Model Comparison Framework

Supervised Learning Framework

Definition: Learn mapping $f : X \to y$ from labeled training data $\{(x_i, y_i)\}_{i=1}^{n}$.

Two Main Tasks:

- **Classification:** y is categorical (e.g., spam/ham)
- **Regression:** y is continuous (e.g., price, temperature)

This Session Focus: Binary and multi-class classification models.

The Supervised Learning Workflow

1. Data Collection
2. Preprocessing
3. Train-Test Split
4. Model Training
5. Hyperparameter Tuning
6. Evaluation
7. Deployment

Iterative: Refine based on evaluation feedback.

Data Preparation Recap

Critical Steps Before Modeling:

- Handle missing values (impute or remove)
- Encode categorical variables (one-hot, label)
- Scale numerical features (for distance-based models)
- Address class imbalance (SMOTE, weighting)
- Split data: train/validation/test

Golden Rule: Apply preprocessing *after* splitting to avoid data leakage.

Evaluation Framework: Metrics

Classification Metrics:

- **Accuracy:** Overall correctness (caution: imbalanced data)
- **Precision:** $\frac{TP}{TP+FP}$ – minimize false positives
- **Recall:** $\frac{TP}{TP+FN}$ – minimize false negatives
- **F1-Score:** Harmonic mean of precision and recall
- **ROC-AUC:** Ranking ability across thresholds

Choose metrics aligned with business cost of errors.

Evaluation Framework: Validation

Why Cross-Validation?

- Single split may give unstable estimates
- CV uses all data for both training and testing
- Provides measure of model variance

k-Fold CV Process:

1. Partition data into k folds
2. For each fold: train on $k-1$, test on 1
3. Average performance across k iterations

Common: $k = 5$ or $k = 10$; stratify for classification.

Model Comparison Strategy

Fair Comparison Principles:

- Use same train/test splits for all models
- Tune hyperparameters on validation set only
- Report multiple metrics (not just accuracy)
- Include confidence intervals via CV
- Consider computational cost and interpretability

Baseline First: Always compare against simple models (e.g., logistic regression).

Handling Class Imbalance

Problem: Minority class underrepresented $\rightarrow$ biased predictions.

Solutions:

- **Resampling:**
  - Oversample minority (SMOTE)
  - Undersample majority
- **Algorithm-Level:**
  - Class weights (e.g., `class_weight` in sklearn)
  - Threshold adjustment post-prediction
- **Metric Choice:** Use F1, AUC instead of accuracy

Feature Scaling: When and Why

Models Requiring Scaling:

- K-Nearest Neighbors (distance-based)
- Support Vector Machines (margin optimization)
- Neural Networks (gradient stability)
- Regularized regression (penalty fairness)

Models Not Requiring Scaling:

- Tree-based methods (RF, XGBoost)
- Naive Bayes (probability-based)

Best Practice: Scale when in doubt; it rarely hurts.

R Setup for Supervised Learning

```r
# Essential packages
library(tidyverse)     # Data manipulation
library(caret)         # Unified ML interface
library(pROC)          # ROC analysis
library(randomForest)  # Random Forest
library(xgboost)       # XGBoost
library(e1071)         # SVM, Naive Bayes
library(MASS)          # LDA, QDA
library(class)         # KNN

# Set seed for reproducibility
set.seed(2026)

# Load example data
data("iris")
data("mtcars")
```

Tip: Use `tidymodels` for modern, consistent workflow.

Logistic Regression: Overview

Purpose: Model probability of binary outcome $y \in \{0, 1\}$.

Key Idea: Transform linear predictor to $[0, 1]$ via logistic function.

Advantages:

- Interpretable coefficients (log-odds)
- Probabilistic output
- Efficient computation
- Baseline for classification tasks

Limitation: Assumes linear decision boundary in log-odds space.

Logistic Regression: Mathematical Foundation

Logistic (Sigmoid) Function:

$$p = P(y = 1|x) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p)}}$$

Log-Odds (Logit) Transformation:

$$\log\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p$$

Estimation: Maximum Likelihood Estimation (MLE)

$$\hat{\beta} = \arg\max_{\beta} \prod_{i=1}^{n} p_i^{y_i} (1 - p_i)^{1-y_i}$$

Interpretation: e^{β_j} = odds ratio for one-unit increase in x_j.

Logistic Regression: Assumptions

- **Binary Outcome:** y must be dichotomous
- **Independence:** Observations independent
- **Linearity in Log-Odds:** $\log(p/(1-p))$ linear in predictors
- **No Perfect Multicollinearity:** Predictors not perfectly correlated
- **Large Sample:** MLE asymptotic properties require adequate n

Diagnostics:

- Hosmer-Lemeshow test (goodness-of-fit)
- Residual analysis (Pearson, deviance)
- Influence measures (Cook's distance)

Logistic Regression: R Implementation

```r
# Train logistic regression
log_model <- glm(admit ~ gpa + gre + rank,
                 data = admissions,
                 family = "binomial")

# View coefficients (log-odds)
summary(log_model)$coefficients

# Predict probabilities
probs <- predict(log_model,
                 newdata = test_data,
                 type = "response")

# Convert to class (threshold = 0.5)
pred_class <- ifelse(probs > 0.5, 1, 0)

# Evaluate
confusionMatrix(factor(pred_class),
                factor(test_data$admit))
```

Tip: Adjust threshold based on precision-recall tradeoff.

Logistic Regression: Interpretation

Coefficient Interpretation:

- $\beta_j > 0$: Increase in $x_j \rightarrow$ higher odds of $y = 1$
- $\beta_j < 0$: Increase in $x_j \rightarrow$ lower odds of $y = 1$
- e^{β_j}: Multiplicative change in odds per unit x_j

Example Output:

- `gre`: $\beta = 0.002$, $OR = e^{0.002} = 1.002$
- Interpretation: Each 1-point GRE increase $\rightarrow$ 0.2% higher odds of admission

Report: Coefficients, odds ratios, 95% CIs, and p-values.

Logistic Regression: Model Assessment

```r
# Pseudo R-squared (McFadden)
null_dev <- log_model$null.deviance
resid_dev <- log_model$deviance
pseudo_r2 <- 1 - (resid_dev / null_dev)

# ROC Curve and AUC
library(pROC)
roc_obj <- roc(test_data$admit, probs)
plot(roc_obj, main = "ROC Curve")
auc(roc_obj)  # Area Under Curve

# Calibration Plot
calibration <- data.frame(
  predicted = probs,
  actual = test_data$admit
)
# Group by predicted deciles and plot mean actual vs predicted
```

Good Model: High AUC (> 0.8), well-calibrated probabilities.

Logistic Regression: When to Use

Use Logistic Regression When:

- Interpretability is important (e.g., healthcare, policy)
- Relationship between features and log-odds is approximately linear
- Dataset is small to moderate size
- Need probabilistic output for decision thresholds
- Baseline model for comparison

Consider Alternatives When:

- Complex non-linear boundaries expected
- High-dimensional feature space
- Interactions dominate without specification

Rule: Start simple; add complexity only if justified.

KNN: Conceptual Overview

Core Idea: "Similar inputs have similar outputs."

Algorithm:

1. For new point x_{new}, find k closest training points
2. Assign class by majority vote (classification)
3. Or average target values (regression)

Key Characteristics:

- **Lazy Learner:** No explicit training; stores all data
- **Non-Parametric:** No assumptions about data distribution
- **Instance-Based:** Prediction depends on local neighborhood

Tradeoff: Simple but computationally expensive at prediction time.

KNN: Distance Metrics

Common Distance Functions:

- **Euclidean:** $d(x, y) = \sqrt{\sum_{j=1}^{p}(x_j - y_j)^2}$
  - Default; sensitive to scale
- **Manhattan:** $d(x, y) = \sum_{j=1}^{p} |x_j - y_j|$
  - Robust to outliers; grid-like paths
- **Minkowski:** $d(x, y) = (\sum_{j=1}^{p} |x_j - y_j|^q)^{1/q}$
  - Generalizes Euclidean ($q = 2$) and Manhattan ($q = 1$)
- **Mahalanobis:** Accounts for feature correlations

Critical: Scale features before computing distances!

KNN: Choosing the Parameter k

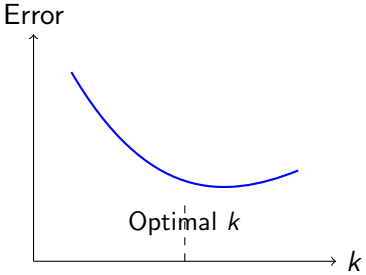
Effect of k:

- **Small k (e.g., 1):** Complex boundary; high variance (overfitting)
- **Large k:** Smooth boundary; high bias (underfitting)
- **Odd k:** Avoids ties in binary classification

Selection Strategy:

1. Use cross-validation to test $k \in \{1, 3, 5, \ldots, \sqrt{n}\}$
2. Plot accuracy vs. k; choose elbow point
3. Consider computational cost (larger k = slower prediction)

KNN: Choosing the Parameter k



KNN: R Implementation

```r
library(class)

# Scale features (critical for KNN)
train_scaled <- scale(train_data[, -target_col])
test_scaled <- scale(test_data[, -target_col],
                     center = attr(train_scaled, "scaled:center"),
                     scale = attr(train_scaled, "scaled:scale"))

# Train and predict (k=5)
preds <- knn(train = train_scaled,
             test = test_scaled,
             cl = train_data$target,
             k = 5,
             prob = TRUE)

# Evaluate
confusionMatrix(preds, test_data$target)

# Find optimal k via CV
k_values <- seq(1, 21, by = 2)
cv_results <- sapply(k_values, function(k) {
  # Cross-validation loop here
```

KNN: Strengths and Limitations

Strengths:

- Simple to understand and implement
- No training phase; adapts easily to new data
- Effective for multi-class problems
- Non-parametric; flexible decision boundaries

Limitations:

- Computationally expensive at prediction ($O(n)$ per query)
- Sensitive to irrelevant features and noise
- Requires feature scaling
- Struggles with high-dimensional data (curse of dimensionality)

Best For: Small to medium datasets with meaningful distance metrics.

KNN: Practical Tips

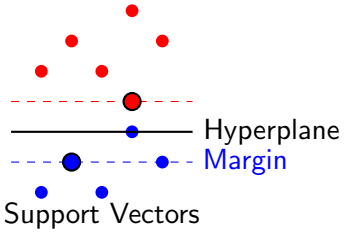
To Improve KNN Performance:

1. **Feature Selection:** Remove irrelevant/noisy features
2. **Dimensionality Reduction:** Apply PCA before KNN
3. **Weighted Voting:** Weight neighbors by inverse distance
4. **Efficient Search:** Use KD-trees or ball trees for large n
5. **Distance Metric Tuning:** Experiment with Manhattan, Minkowski

Rule of Thumb: If $p \gg n$, KNN likely to underperform; consider regularization or feature selection first.

SVM: Conceptual Overview

Core Idea: Find hyperplane that maximizes margin between classes.



Key Term: Support vectors = data points on margin boundaries.

SVM: Mathematical Foundation (Linear)

Primal Optimization Problem:

$$\min_{\beta,\beta_0} \frac{1}{2}\|\beta\|^2 \quad \text{subject to} \quad y_i(\beta^T x_i + \beta_0) \geq 1 \quad \forall i$$

Key Components:

- $\|\beta\|^2$: Margin width (maximize $\rightarrow$ minimize norm)
- Constraint: All points correctly classified with margin ≥ 1
- Solution depends only on support vectors

Soft Margin (Allow Misclassification):

$$\min_{\beta,\beta_0,\xi} \frac{1}{2}\|\beta\|^2 + C\sum_{i=1}^{n} \xi_i$$

where $\xi_i \geq 0$ are slack variables; C controls tradeoff.

Dual Form: Enables kernel trick for non-linear boundaries.

SVM: Kernel Functions

Kernel Trick: Map data to higher dimension without explicit computation.

Common Kernels:

- **Linear:** $K(x, x') = x^T x'$
  - For linearly separable data
- **Polynomial:** $K(x, x') = (1 + x^T x')^d$
  - Captures feature interactions up to degree d
- **RBF (Gaussian):** $K(x, x') = \exp(-\gamma \|x - x'\|^2)$
  - Most popular; flexible non-linear boundaries
- **Sigmoid:** $K(x, x') = \tanh(\alpha x^T x' + c)$
  - Similar to neural network activation

Tune: γ (RBF width), C (regularization), d (polynomial degree).

SVM: Hyperparameters

- C **(Regularization):**
  - Large C: Hard margin (few misclassifications, risk overfitting)
  - Small C: Soft margin (more tolerance, smoother boundary)
- γ **(RBF Kernel Width):**
  - Large γ: Complex boundary (local influence, overfitting)
  - Small γ: Smooth boundary (global influence, underfitting)
- **Kernel Type:** Linear, polynomial, RBF, sigmoid
- **Degree** d**:** For polynomial kernel only

Tuning Strategy: Grid search with cross-validation; start with RBF + default $\gamma = 1/p$.

SVM: R Implementation

```r
library(e1071)

# Scale features (important for SVM)
train_scaled <- scale(train_data[, -target_col])
test_scaled <- scale(test_data[, -target_col],
                     center = attr(train_scaled, "scaled:center"),
                     scale = attr(train_scaled, "scaled:scale"))

# Train SVM with RBF kernel
svm_model <- svm(target ~ .,
                 data = as.data.frame(cbind(train_scaled, target = train_data$target)),
                 kernel = "radial",
                 cost = 1,       # C parameter
                 gamma = 0.1)    # gamma parameter

# Predict
preds <- predict(svm_model,
                 newdata = as.data.frame(test_scaled))

# Evaluate
confusionMatrix(preds, test_data$target)
```

SVM: Strengths and Limitations

Strengths:

- Effective in high-dimensional spaces
- Memory efficient (uses support vectors only)
- Versatile via kernel functions
- Strong theoretical guarantees (maximal margin)

Limitations:

- Computationally intensive for large n ($O(n^2)$ to $O(n^3)$)
- Requires careful tuning of C, γ, kernel
- Probabilistic output requires Platt scaling (not native)
- Less interpretable than linear models

Best For: Small to medium datasets with clear margin separation.

SVM: When to Use

Use SVM When:

- Number of features p is large relative to samples n
- Decision boundary is expected to be non-linear but smooth
- Dataset is small to moderate size ($n < 10,000$)
- Need strong generalization with limited data

Consider Alternatives When:

- Very large datasets (use linear SVM or tree-based methods)
- Need probabilistic outputs with calibration
- Interpretability is critical
- Data is noisy with overlapping classes

Practical Tip: Always scale features and start with RBF kernel + grid search.

Naive Bayes: Bayes' Theorem Foundation

Bayes' Theorem:

$$P(y|x) = \frac{P(x|y) \cdot P(y)}{P(x)}$$

Components:

- $P(y|x)$: Posterior probability (what we want)
- $P(x|y)$: Likelihood (probability of features given class)
- $P(y)$: Prior probability (class frequency in training)
- $P(x)$: Evidence (normalizing constant; same for all classes)

Classification Rule:

$$\hat{y} = \arg\max_{y} P(y) \prod_{j=1}^{p} P(x_j|y)$$

Key: We only need to compute numerator; denominator cancels in comparison.

Naive Bayes: The "Naive" Assumption

Conditional Independence:

$$P(x|y) = P(x_1, x_2, \ldots, x_p|y) = \prod_{j=1}^{p} P(x_j|y)$$

Implications:

- Features are assumed independent given the class
- Simplifies computation: estimate p univariate distributions instead of one p-variate
- Often violated in practice, yet model performs well empirically

Why It Works Despite Violation:

- Classification depends on relative probabilities, not absolute
- Bias from independence assumption may cancel across classes
- Low variance due to simple parameter estimation

Result: Fast, scalable, surprisingly effective baseline.

Naive Bayes: Distribution Assumptions

Common Feature Distributions:

- **Gaussian NB:** Continuous features

$$P(x_j|y) = \frac{1}{\sqrt{2\pi\sigma_{yj}^2}} \exp\left(-\frac{(x_j - \mu_{yj})^2}{2\sigma_{yj}^2}\right)$$

- **Multinomial NB:** Count data (e.g., word frequencies)

$$P(x_j|y) \propto \theta_{yj}^{x_j}, \quad \theta_{yj} = \frac{N_{yj} + \alpha}{N_y + \alpha p}$$

- **Bernoulli NB:** Binary features (presence/absence)

$$P(x_j|y) = \theta_{yj}^{x_j}(1 - \theta_{yj})^{1-x_j}$$

Smoothing: Laplace smoothing ($\alpha > 0$) prevents zero probabilities.

Naive Bayes: R Implementation

```r
library(e1071)

# Gaussian Naive Bayes (default for continuous features)
nb_model <- naiveBayes(target ~ ., data = train_data)

# View learned parameters
nb_model$apriori       # P(y): class priors
nb_model$tables        # P(x_j|y): conditional distributions

# Predict class and probabilities
preds <- predict(nb_model, newdata = test_data)
probs <- predict(nb_model, newdata = test_data, type = "raw")

# Evaluate
confusionMatrix(preds, test_data$target)

# For text data: use multinomial NB with tm or quanteda
# Preprocess: tokenize, create document-term matrix, then train
```

Note: No hyperparameters to tune (except smoothing α); very fast to train.

Naive Bayes: Strengths and Limitations

Strengths:

- Extremely fast training and prediction ($O(np)$)
- Handles high-dimensional data well (e.g., text)
- Robust to irrelevant features
- Works well with small training sets
- Provides probabilistic output

Limitations:

- Strong independence assumption rarely holds
- Cannot learn feature interactions
- Probability estimates may be poorly calibrated
- Sensitive to feature distribution misspecification

Best For: Text classification, baseline model, high-dimensional sparse data.

Naive Bayes: Practical Applications

Common Use Cases:

- **Spam Detection:** Word presence/absence features
- **Sentiment Analysis:** Bag-of-words representation
- **Document Categorization:** News topics, medical records
- **Real-time Prediction:** Low-latency requirements

Tips for Better Performance:

1. Use appropriate distribution (Gaussian vs. Multinomial)
2. Apply feature selection to remove noisy features
3. Tune smoothing parameter α via cross-validation
4. Combine with TF-IDF weighting for text data
5. Calibrate probabilities post-hoc if needed (Platt scaling)

Rule: Excellent baseline; hard to beat for text with limited data.

LDA: Statistical Foundation

Assumptions:

1. Features X follow multivariate normal distribution within each class
2. All classes share the same covariance matrix: $\Sigma_k = \Sigma \quad \forall k$

Discriminant Function (for class k):

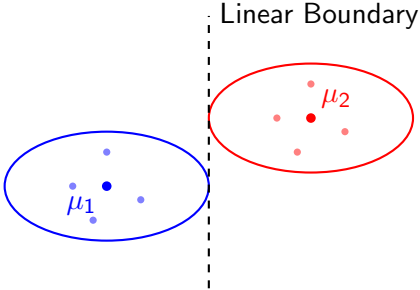
$$\delta_k(x) = x^T \Sigma^{-1} \mu_k - \frac{1}{2} \mu_k^T \Sigma^{-1} \mu_k + \log(\pi_k)$$

Classification Rule:

$$\hat{y} = \arg\max_k \delta_k(x)$$

Key Insight: Decision boundaries are *linear* (hence "Linear" Discriminant Analysis).

LDA: Geometric Interpretation



Why Linear? Shared $\Sigma \rightarrow$ quadratic terms cancel in $\delta_k(x) - \delta_l(x)$.
Estimation: μ_k, Σ, π_k from training data via MLE.

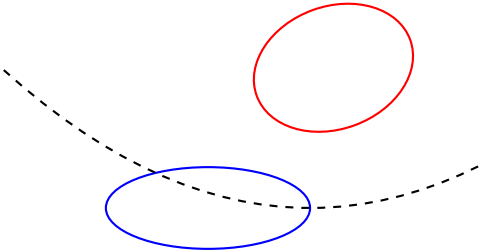
QDA: Relaxing the Covariance Assumption

QDA Assumption: Each class has its own covariance matrix Σ_k.

Quadratic Discriminant Function:

$$\delta_k(x) = -\frac{1}{2}\log|\Sigma_k| - \frac{1}{2}(x-\mu_k)^T\Sigma_k^{-1}(x-\mu_k) + \log(\pi_k)$$

Consequence: Decision boundaries are *quadratic* (curved).



Quadratic Boundary

Tradeoff: More flexible but requires estimating more parameters ($K \times p(p+1)/2$).

LDA vs. QDA: Comparison

Criterion	LDA	QDA
Covariance	Shared: $\Sigma_k = \Sigma$	Class-specific: Σ_k
Boundary	Linear	Quadratic
Parameters	$Kp + p(p+1)/2$	$Kp + K \cdot p(p+1)/2$
Bias-Variance	Higher bias, lower variance	Lower bias, higher variance
Sample Size	Works with smaller n	Requires larger n per class
When to Use	Classes have similar spread; n limited	Classes have different shapes; ample data

Practical Tip: Start with LDA; switch to QDA if residual analysis suggests unequal covariances.

LDA/QDA: R Implementation

```r
library(MASS)
# Linear Discriminant Analysis
lda_model <- lda(target ~ ., data = train_data)
# Predict class and posterior probabilities
lda_preds <- predict(lda_model, newdata = test_data)
lda_class <- lda_preds$class
lda_probs <- lda_preds$posterior
# Quadratic Discriminant Analysis
qda_model <- qda(target ~ ., data = train_data)
qda_preds <- predict(qda_model, newdata = test_data)
# Evaluate both
confusionMatrix(lda_class, test_data$target)
confusionMatrix(qda_preds$class, test_data$target
# Compare decision boundaries (2D example)
# Use contour plots of posterior probabilities
```

Diagnostic: Check equality of covariance matrices with Box's M test (caution: sensitive to normality).

LDA/QDA: When to Use

Use LDA When:

- Classes have similar covariance structures
- Sample size is limited relative to features
- Interpretability of linear boundaries is valuable
- Need stable estimates with small n

Use QDA When:

- Classes have distinctly different spreads/orientations
- Sufficient data to estimate class-specific covariances
- Non-linear (quadratic) boundaries are expected
- Willing to trade some stability for flexibility

Extension: Regularized Discriminant Analysis (RDA) interpolates between LDA and QDA.

Ensemble Learning: Bagging Concept

Bagging (Bootstrap Aggregating):

1. Create B bootstrap samples from training data
2. Train base model (e.g., decision tree) on each sample
3. Aggregate predictions:
   - Classification: Majority vote
   - Regression: Average

Why Bagging Works:

- Reduces variance of unstable learners (e.g., deep trees)
- Bootstrap sampling introduces diversity among models
- Aggregation smooths out individual model errors

Key: Base learners should be *unbiased but high-variance*.

Random Forest: Core Idea

Random Forest = Bagging + Feature Randomness

Algorithm:

1. For $b = 1$ to B:
   1. Draw bootstrap sample of training data
   2. Grow tree T_b: at each split, consider only m_{try} random features
   3. Grow tree to maximum depth (no pruning)
2. Aggregate predictions across all B trees

Feature Randomness: Reduces correlation among trees $\rightarrow$ lower ensemble variance.

Random Forest: Mathematical Insight

Variance Reduction via Decorrelation:
Ensemble variance:

$$\mathrm{Var}(\bar{f}) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$$

where ρ = average correlation between trees, σ^2 = individual tree variance.

Effect of m_{try}:

- Small m_{try}: Trees more diverse ($\rho \downarrow$) but weaker individually ($\sigma^2 \uparrow$)
- Large m_{try}: Trees more similar ($\rho \uparrow$) but stronger individually ($\sigma^2 \downarrow$)
- Optimal: Balance (default $m_{try} = \sqrt{p}$ for classification)

Result: RF achieves low bias (deep trees) + low variance (decorrelation).

Random Forest: Feature Importance

Two Common Measures:

- **Mean Decrease in Impurity (MDI):**
  - Sum of impurity reduction (Gini/entropy) from splits on feature j
  - Fast to compute; biased toward high-cardinality features
- **Permutation Importance:**
  - Randomly shuffle feature j in OOB samples; measure accuracy drop
  - More reliable; accounts for feature interactions

Out-of-Bag (OOB) Error:

- Each tree trained on 63% of data; remaining 37% = OOB sample
- Aggregate OOB predictions $\rightarrow$ unbiased error estimate (no CV needed)

Use: Permutation importance for reliable feature selection.

Random Forest: Hyperparameters

- n_{tree} **(Number of Trees):**
  - Larger = more stable; diminishing returns after 500
  - Tune via OOB error curve
- m_{try} **(Features per Split):**
  - Classification: $\sqrt{p}$ (default); Regression: $p/3$
  - Smaller values increase diversity
- **Tree Depth / Node Size:**
  - `max.depth`: Control overfitting (default: grow to purity)
  - `nodesize`: Min samples per leaf (default: 1 for classification)
- **Sampling:** Bootstrap (default) or subsampling

Tuning Priority: m_{try} ¿ node size ¿ n_{tree}.

Random Forest: R Implementation

```r
library(randomForest)
# Train Random Forest
rf_model <- randomForest(target ~ .,
                         data = train_data,
                         ntree = 500,       # number of trees
                         mtry = 3,          # features per split
                         nodesize = 5,      # min leaf size
                         importance = TRUE) # compute feature importance
# OOB error estimate
rf_model$err.rate  # Plot to see convergence
# Predict
preds <- predict(rf_model, newdata = test_data)
# Feature importance (permutation)
importance(rf_model, type = 1)
varImpPlot(rf_model)  # Visualize
# Evaluate
confusionMatrix(preds, test_data$target)
```

Tip: Use `cforest` (party package) for unbiased variable selection with correlated features.

Random Forest: Strengths and Limitations

Strengths:

- High accuracy with minimal tuning
- Handles non-linearities and interactions automatically
- Robust to outliers and irrelevant features
- Provides feature importance and OOB error
- Works well with mixed data types

Limitations:

- Less interpretable than single trees
- Can overfit noisy datasets (though rare)
- Slower prediction than linear models (but parallelizable)
- Extrapolation poor (bounded by training range)

Best For: Tabular data with complex patterns; strong baseline for many tasks.

Boosting: Conceptual Overview

Core Idea: Sequentially train weak learners, each correcting predecessors.

Algorithm Outline:

1. Initialize model $F_0(x)$ (e.g., mean target)
2. For $m = 1$ to M:
   1. Compute pseudo-residuals: $r_{im} = -\left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)}\right]$
   2. Fit weak learner $h_m(x)$ to residuals r_{im}
   3. Update: $F_m(x) = F_{m-1}(x) + \nu \cdot h_m(x)$
3. Output final model $F_M(x)$

Key: ν (learning rate) controls contribution of each tree; small ν = more robust.

XGBoost: Mathematical Foundation

Objective Function (Regularized):

$$\mathcal{L} = \sum_{i=1}^{n} L(y_i, \hat{y}_i) + \sum_{k=1}^{K} \Omega(f_k)$$

where $\Omega(f) = \gamma T + \frac{1}{2}\lambda \|\mathbf{w}\|^2$ penalizes tree complexity.

Second-Order Approximation:

$$\mathcal{L}^{(t)} \approx \sum_{i=1}^{n} \left[g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i) \right] + \Omega(f_t)$$

with $g_i = \partial_{\hat{y}} L$, $h_i = \partial_{\hat{y}}^2 L$ (gradient and Hessian).

Optimal Leaf Weight:

$$w_j^* = -\frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda}$$

Advantage: Uses second-order information for faster convergence vs. gradient boosting.

XGBoost: Key Features and Regularization

Regularization Parameters:

- λ **(L2):** Penalizes large leaf weights (smoothness)
- α **(L1):** Encourages sparse leaf weights (feature selection)
- γ **(min_split_loss):** Min loss reduction to make a split (pruning)

Other Critical Hyperparameters:

- **Learning Rate (η):** Step size shrinkage (typical: 0.01–0.3)
- **Max Depth:** Controls tree complexity (typical: 3–10)
- **Subsample / Colsample_bytree:** Row/column sampling for stochasticity
- **Min Child Weight:** Min sum of Hessian in child (prevents overfitting)

Tuning Strategy: Start with small η, large n_{rounds}; tune depth and regularization.

XGBoost: R Implementation

```r
library(xgboost)
# Prepare data matrix (required format)
dtrain <- xgb.DMatrix(data = as.matrix(train_data[, -target_col]),
                      label = train_data$target)
dtest <- xgb.DMatrix(data = as.matrix(test_data[, -target_col]))
# Set parameters
params <- list(
  objective = "binary:logistic",  # or "multi:softprob"
  eval_metric = "logloss", max_depth = 6, eta = 0.1, # learning rate
  subsample = 0.8, colsample_bytree = 0.8,
  lambda = 1, # L2 regularization
  alpha = 0           # L1 regularization
)
# Train with early stopping
xgb_model <- xgb.train(params = params, data = dtrain, nrounds = 500,
                       watchlist = list(train = dtrain, test = dtest),
                       early_stopping_rounds = 20, verbose = 1)
# Predict
preds <- predict(xgb_model, dtest)
pred_class <- ifelse(preds > 0.5, 1, 0)
```

Tip: Use `xgb.cv()` for cross-validated hyperparameter tuning.

XGBoost: Strengths and Best Practices

Strengths:

- State-of-the-art accuracy on tabular data
- Efficient implementation (parallel, cache-aware)
- Built-in regularization reduces overfitting
- Handles missing values natively
- Provides feature importance

Best Practices:

1. Start with conservative depth (3–6) and small learning rate (0.01–0.1)
2. Use early stopping to determine optimal n_{rounds}
3. Tune γ, λ, α to control complexity
4. Validate with stratified CV for imbalanced data
5. Monitor train/test loss curves for overfitting signs

Caution: Powerful but prone to overfitting if not regularized; always use validation.

Summary, Resources, and Q&A

Key Takeaways:

- Supervised learning maps features to labels via trained models
- Model choice depends on data size, feature type, interpretability needs
- Logistic Regression: interpretable baseline; KNN: simple, distance-based
- SVM: maximal margin with kernels; Naive Bayes: fast, probabilistic
- LDA/QDA: Gaussian assumptions, linear/quadratic boundaries
- Random Forest: bagging + feature randomness; XGBoost: sequential boosting with regularization
- Always validate with proper metrics and cross-validation

Further Learning:

- *An Introduction to Statistical Learning* (Ch. 4, 8, 9) – Free online
- *Elements of Statistical Learning* (Hastie et al.) – Advanced theory
- CDAM Resources: `https://cdam.chuka.ac.ke`
- R Packages: `tidymodels`, `caret`, `xgboost`, `randomForest`

Thank you! Questions?